



# KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

## Chương 2: Biểu diễn kiến trúc và thành phần

ThS. Nguyễn Văn Hữu Hoàng

# I NỘI DUNG

Nội dung của chương này tập trung xem xét một số chủ đề sau đây:

- 2.1. Kiến trúc và các kiểu kiến trúc
- 2.2. Mô hình hóa kiến trúc
- 2.3. Kiến trúc phần mềm dựa trên thành phần
- 2.4. Biểu diễn kiến trúc và thành phần với UML

## I 2.1 KIẾN TRÚC VÀ CÁC KIỂU KIẾN TRÚC

Tiến trình thiết kế kiến trúc thông thường gồm các bước cơ bản sau đây:

1. Hiểu được vấn đề cần giải quyết
2. Xác định các phần tử của thiết kế và quan hệ của chúng
3. Đánh giá thiết kế kiến trúc
4. Biến đổi thiết kế kiến trúc

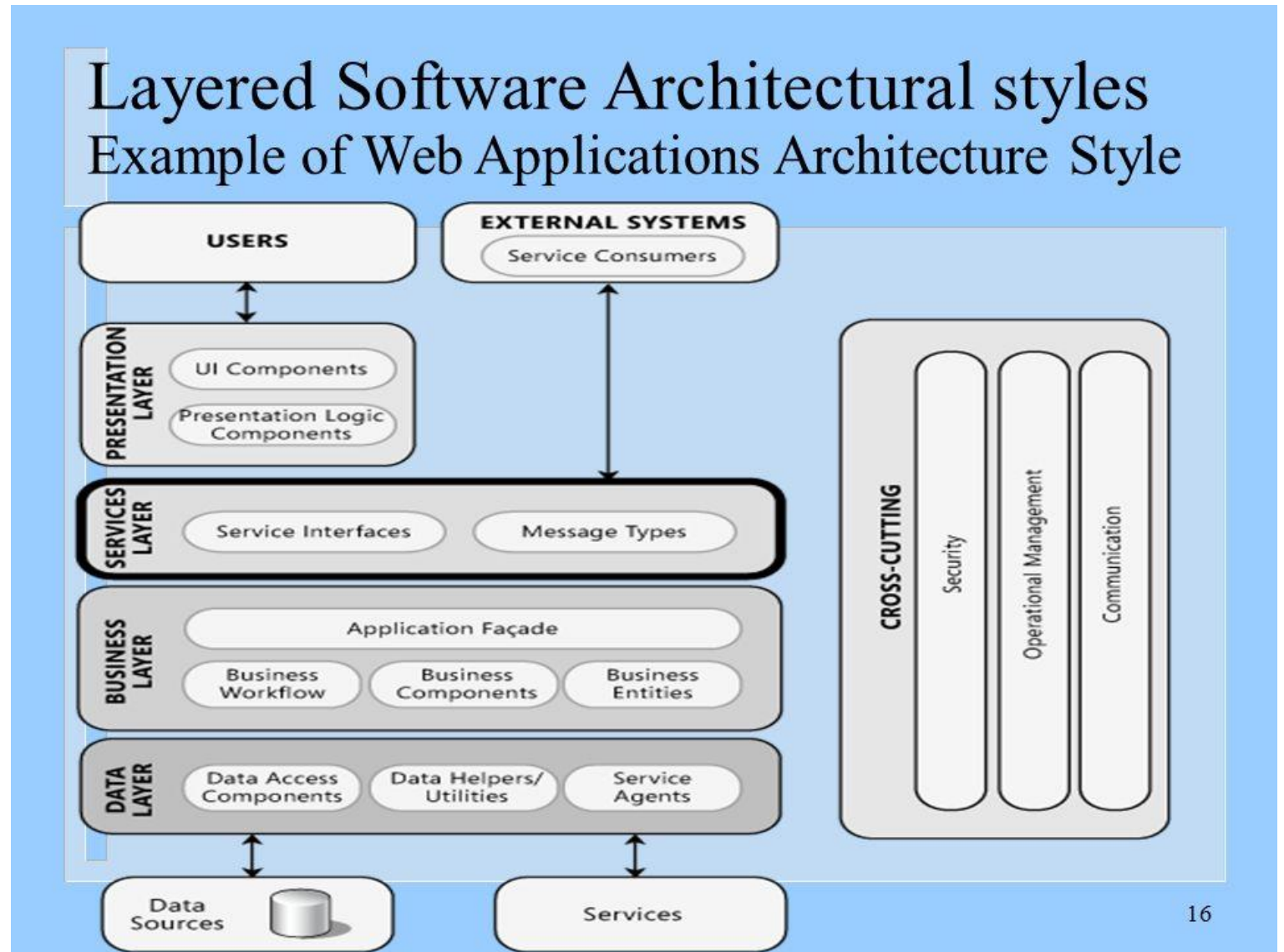
*Mục đích của thiết kế kiến trúc là tập trung vào phân rã hệ thống thành các thành phần và tương tác giữa các thành phần sao cho thỏa mãn các yêu cầu chức năng và phi chức năng được đặt ra.*

# I KIẾN TRÚC PHẦN MỀM TỔNG QUÁT

- **Tầng trình diễn** (presentation layer): Bao gồm các thành phần giao diện người dùng (UI: User Interface) và các thành phần logic trình diễn (Presentation logic).
- **Tầng dịch vụ** (service layer): Bao gồm các giao diện dịch vụ và các kiểu dịch vụ.
- **Tầng nghiệp vụ** (business layer): Bao gồm các thành phần, thực thể nghiệp vụ và luồng hoạt động nghiệp vụ.
- **Tầng dữ liệu** (data layer): Bao gồm các thành phần truy nhập dữ liệu, các tiện ích dữ liệu và các tác nhân dịch vụ.
- **Tầng kết nối** (cross cutting): Bao gồm an toàn, truyền thông và quản lý điều hành.

# KIẾN TRÚC PHẦN MỀM TỔNG QUÁT

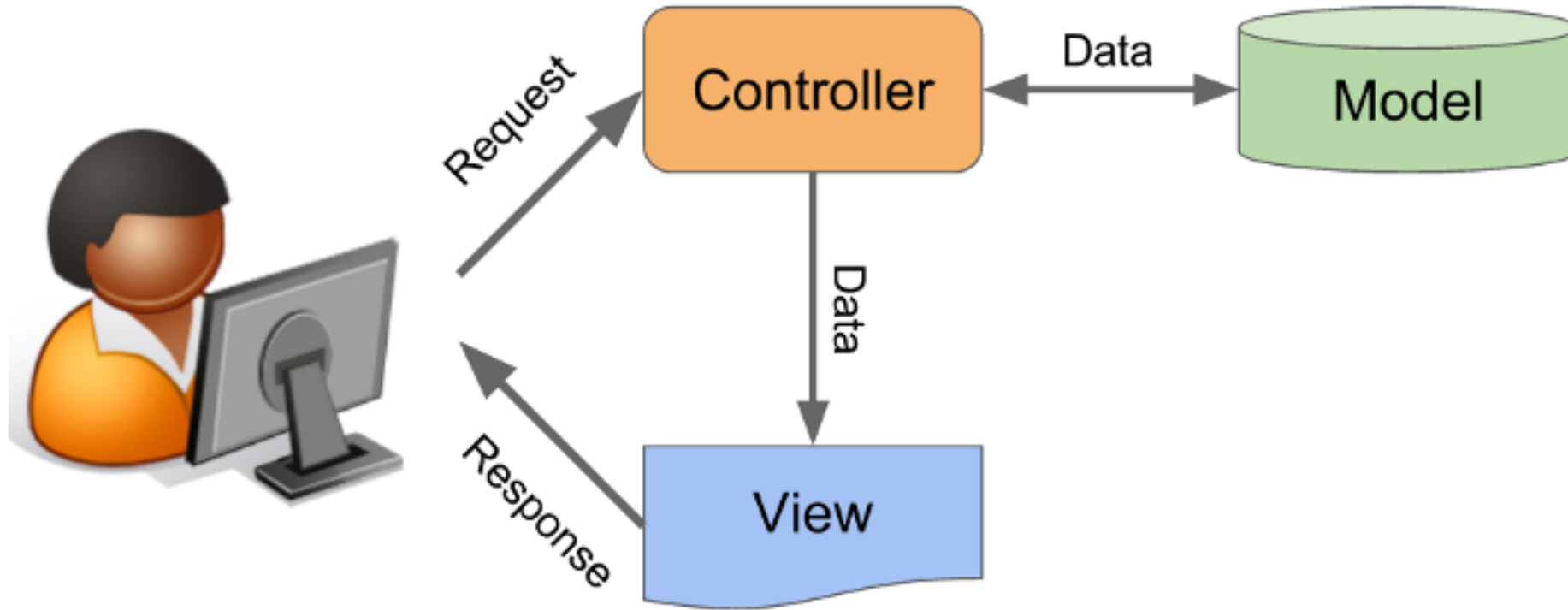
---



# I CÁC NGUYÊN LÝ THIẾT KẾ CƠ BẢN

- **Nguyên lý phân rã các chức năng:** Phân chia ứng dụng thành những nhóm sao cho tối thiểu hóa được sự chồng lấp các chức năng, mục tiêu đạt được *kết hợp cao (high cohesion)* và *kết dính thấp (low coupling)*.
- **Nguyên lý đơn nhiệm:** Mỗi thành phần hay môđun nên chịu trách nhiệm cho chỉ một chức năng hay một hợp chức năng có liên kết nhau.
- **Nguyên lý hiểu biết tối thiểu:** Một thành phần hay một đối tượng không nên biết về chi tiết bên trong của các thành phần hay các đối tượng khác.
- **Nguyên lý không tự lặp lại:** Chỉ nên đặc tả ý định tại một vị trí.
- **Nguyên lý cực tiểu hóa thiết kế:** Chỉ thiết kế những gì cần thiết.

# I VÍ DỤ MÔ HÌNH MVC



**Model (dữ liệu):** Quản lý xử lý các dữ liệu.

**View (giao diện):** Nơi hiển thị dữ liệu cho người dùng.

**Controller (bộ điều khiển):** Điều khiển sự tương tác của hai thành phần Model và View.

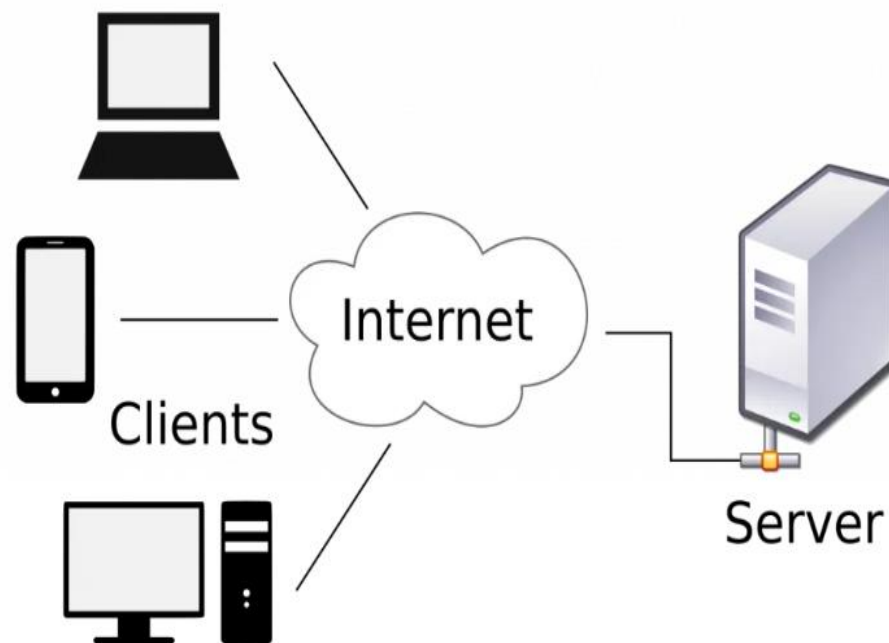
# | CÁC KIỂU KIẾN TRÚC

| Phân loại  | Kiểu kiến trúc                                           |
|------------|----------------------------------------------------------|
| Giao tiếp  | Kiến trúc hướng dịch vụ, Kiến trúc bus thông điệp        |
| Triển khai | Client/Server, N-tầng, 3-Tầng                            |
| Miền       | Thiết kế hướng miền                                      |
| Cấu trúc   | Dựa vào thành phần, Hướng đối tượng, Kiến trúc theo tầng |



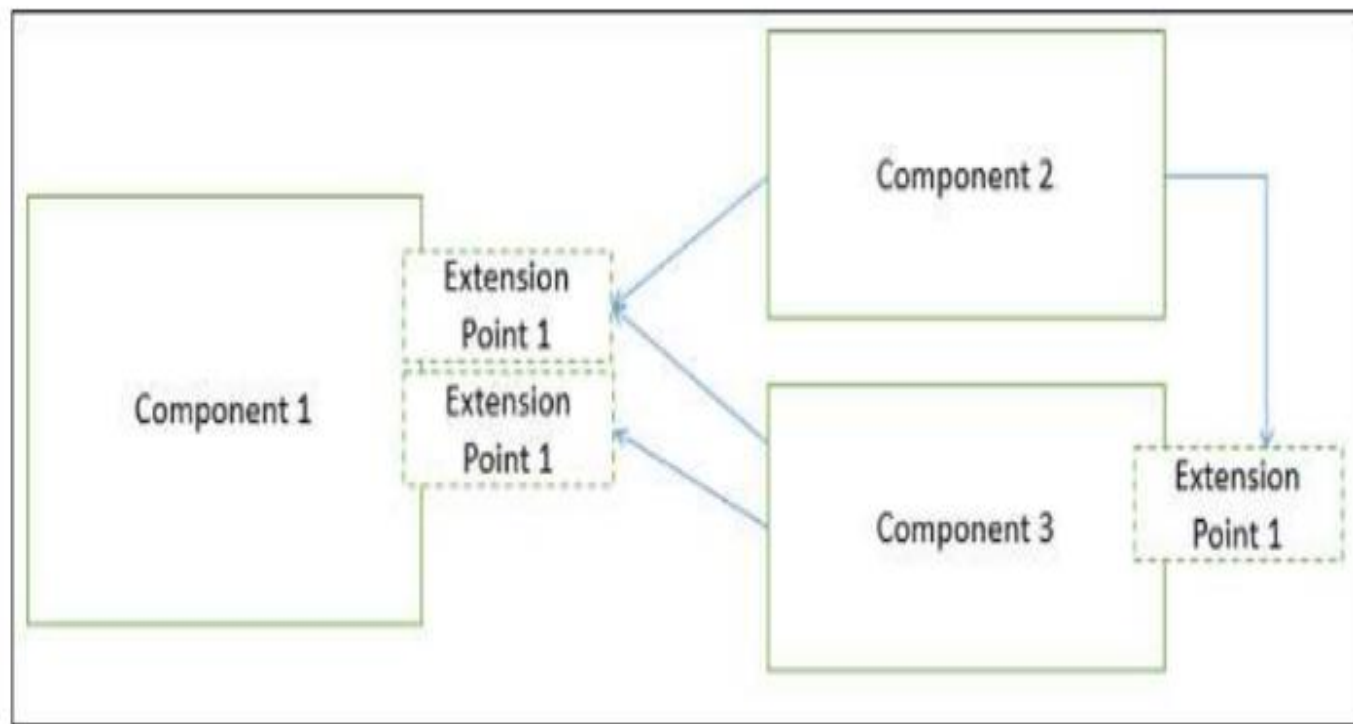
# | Kiểu kiến trúc Client/Server

- Kiến trúc client/server mô tả hệ thống phân tán bao gồm các hệ thống máy chủ, các máy khách và các mạng lưới kết nối.
- Ưu điểm:
  - Có tính bảo mật cao.
  - Truy cập dữ liệu tập trung.
  - Dễ dàng bảo trì.
- Nhược điểm:
  - Dễ tắc nghẽn lưu lượng.
  - Độ bền phụ thuộc vào server.
  - Chi phí bảo trì khá cao.



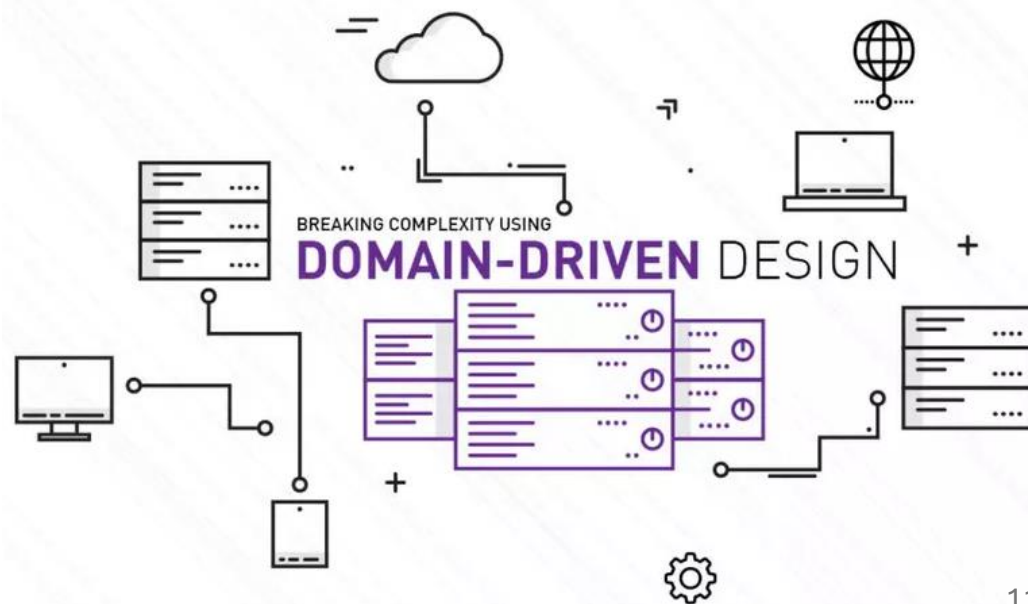
# | Kiểu kiến trúc dựa vào thành phần

- Phân rã kiến trúc thành các thành phần chức năng riêng biệt hoặc các thành phần tiếp xúc với giao diện chuẩn qua kết nối.
- Nguyên tắc của thiết kế thành phần:
  - Khả năng tái sử dụng.
  - Khả năng thay thế được.
  - Khả năng mở rộng.
  - Được đóng gói.
  - Tính độc lập.
- Ưu điểm:
  - Dễ triển khai.
  - Chi phí thấp hơn.
  - Dễ phát triển.
  - Tái sử dụng.
  - Giảm sự phức tạp kỹ thuật.



# | Kiểu kiến trúc hướng miền

- Là một cách tiếp cận hướng đối tượng để thiết kế phần mềm dựa trên các hoạt động nghiệp vụ với các thuộc tính, hành vi và quan hệ giữa chúng.
- Để áp dụng thiết kế dựa trên mô hình miền, chúng ta phải có một hiểu biết khá rõ về lĩnh vực nghiệp vụ mà mình muốn mô hình, hoặc có kỹ năng cũng như kiến thức nghiệp vụ liên quan.
- Ưu điểm:
  - Truyền thông liên lạc tốt.
  - Dễ dàng mở rộng.
  - Dễ dàng kiểm chứng các thành phần.

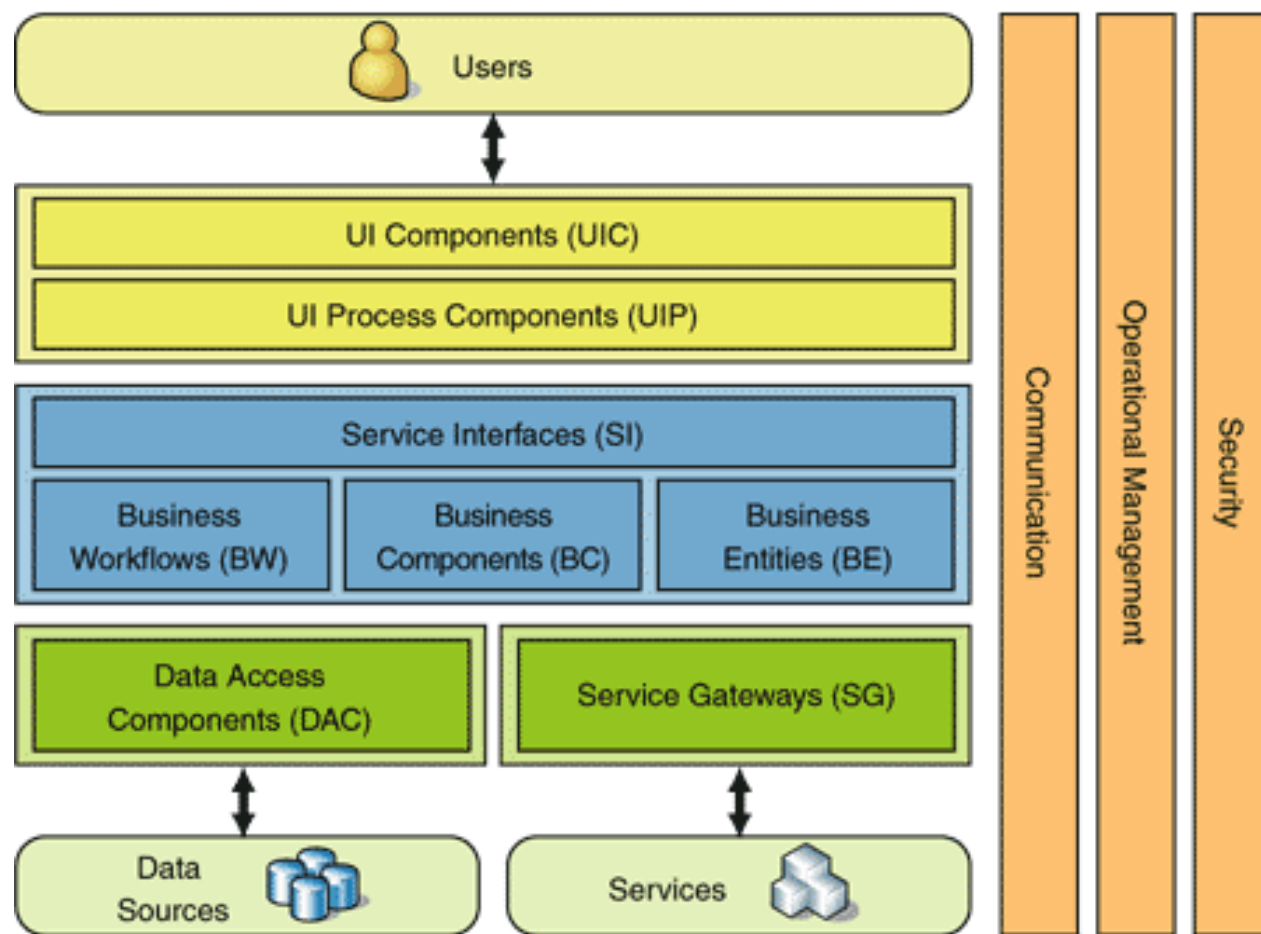


## | Kiểu kiến trúc phân tầng

- Các nhóm chức năng có liên quan trong một ứng dụng thành các tầng riêng biệt được xếp chồng lên nhau theo chiều dọc.
- Các tầng của một ứng dụng có thể nằm trên các máy tính vật lý như nhau hoặc có thể được phân phối trên các máy tính riêng biệt.
- Nguyên tắc thiết kế phân tầng:
  - Trừu tượng.
  - Đóng gói.
  - Xác định rõ ràng các tầng chức năng.
  - Kết dính cao.
  - Tái sử dụng.
  - Kết nối lỏng lẻo.
- Ví dụ mô hình MVC là kiểu kiến trúc phân tầng.

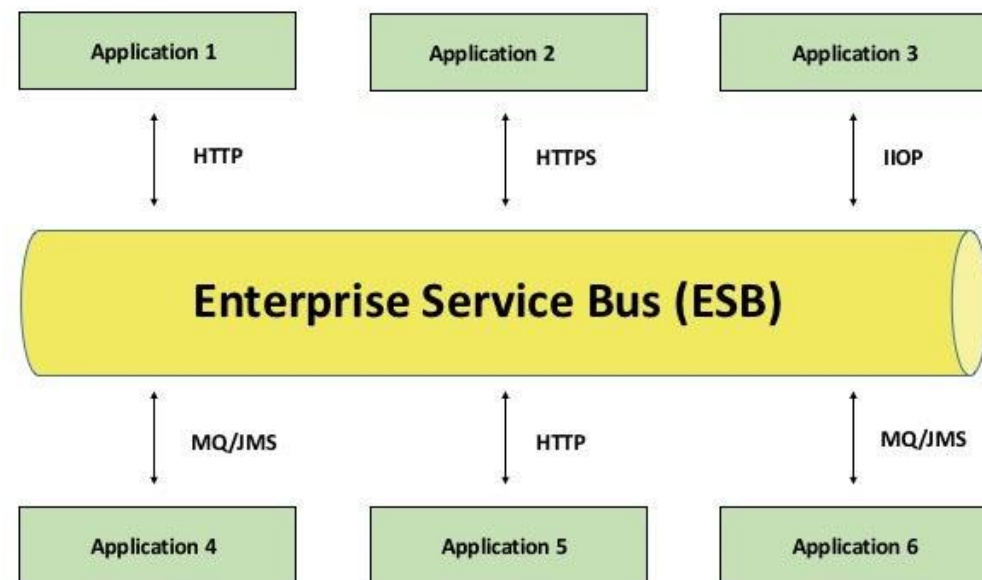
# | Kiểu kiến trúc phân tầng

- Ưu điểm:
  - Trừu tượng.
  - Cô lập các tầng.
  - Dễ dàng quản lý.
  - Cải thiện hiệu suất.
  - Dễ kiểm thử.



## | Kiểu kiến trúc bus thông điệp

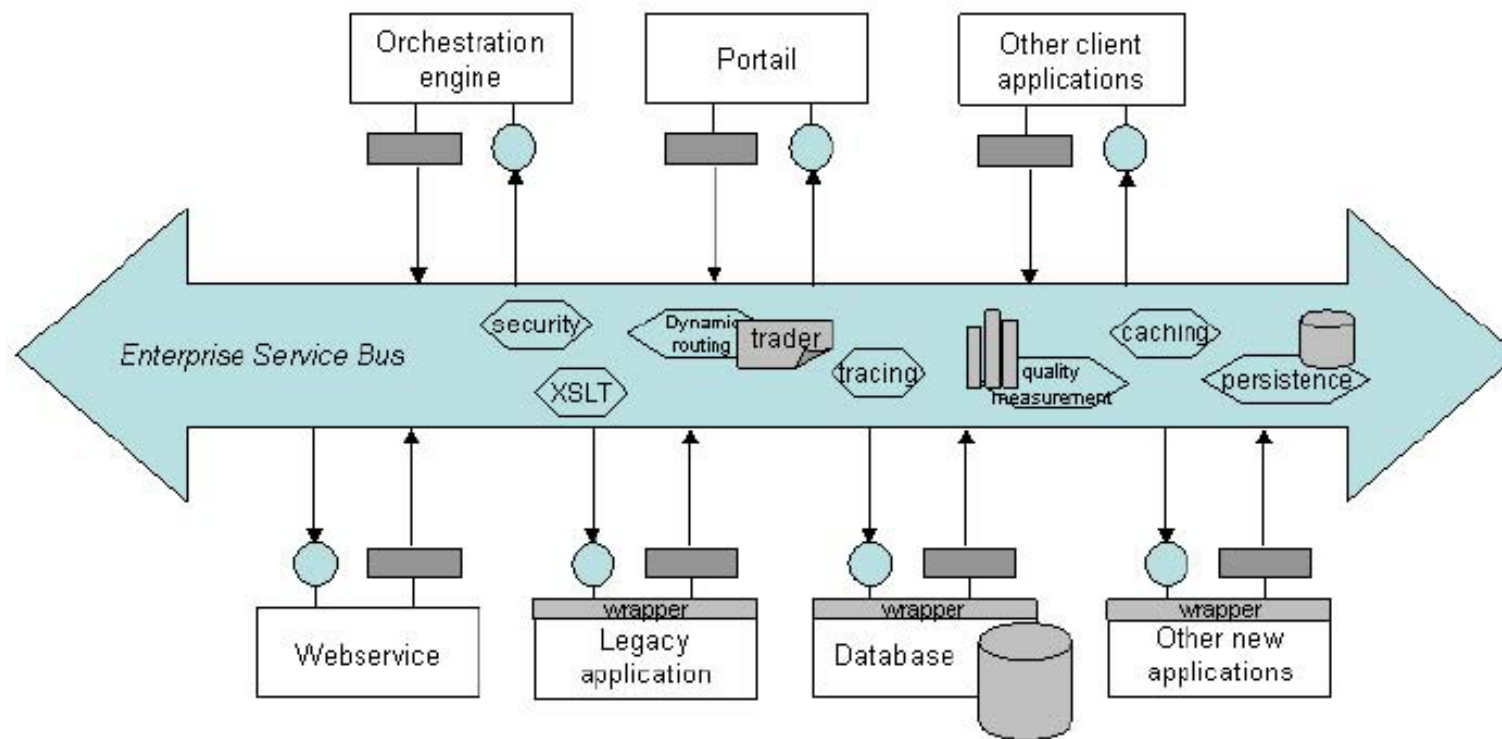
- Kiến trúc bus thông điệp mô tả các nguyên tắc sử dụng một hệ thống phần mềm có thể nhận và gửi thông điệp qua một hoặc nhiều kênh truyền thông
- Một bus thông thường cung cấp khả năng xử lý :
  - Giao tiếp hướng thông điệp.
  - Xử lý logic phức tạp.
  - Chỉnh sửa cho xử lý logic.
  - Kết hợp nhiều môi trường khác nhau (Java, .NET ...).



# | Kiểu kiến trúc bus thông điệp

- Ưu điểm:

- Khả năng mở rộng cao.
- Ít phức tạp.
- Linh hoạt.
- Kết nối lỏng.
- Ứng dụng đơn giản.



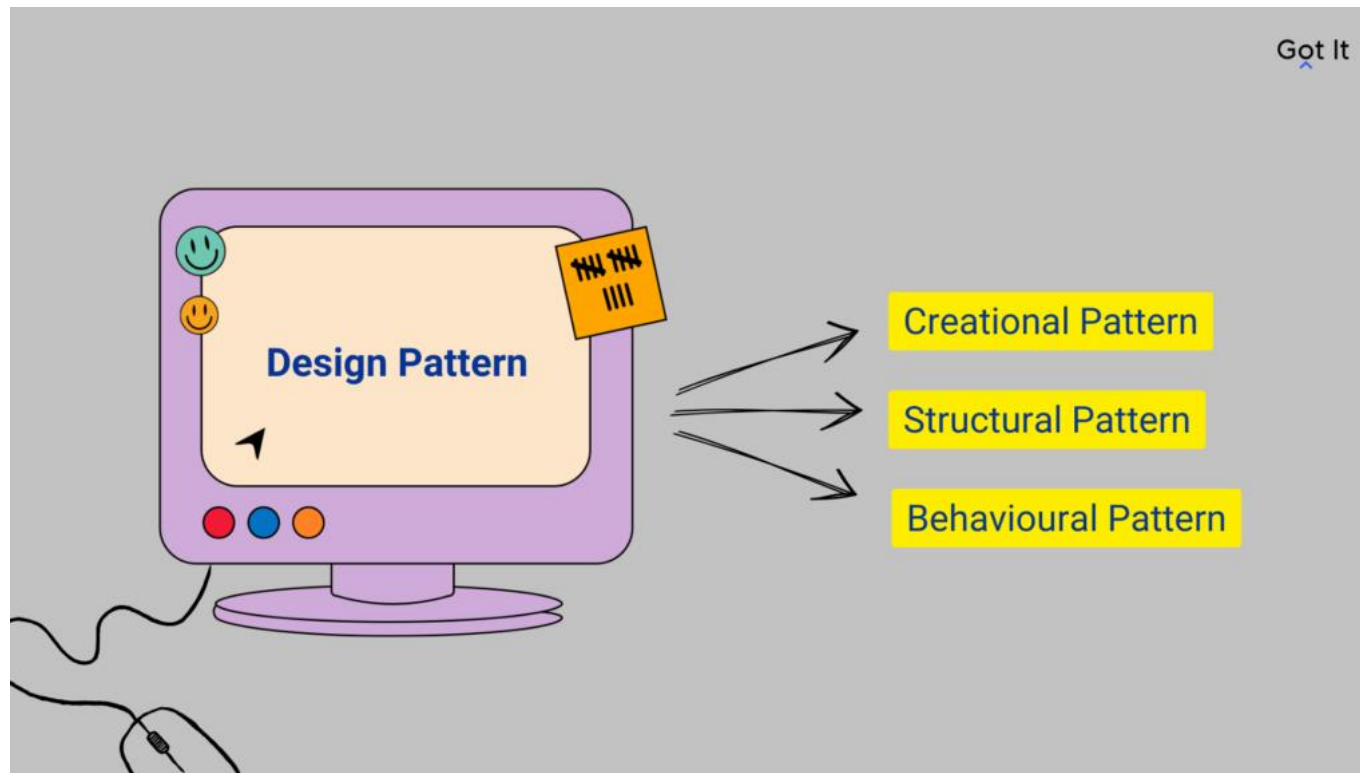
# | Kiểu kiến trúc hướng đối tượng

- Kiến trúc hướng đối tượng (Design Pattern) là một mô hình thiết kế dựa trên sự phân chia trách nhiệm một ứng dụng hoặc hệ thống cho các thành phần tái sử dụng.
- Đối tượng là rời rạc, độc lập, và liên kết lỏng lẻo. Chúng giao tiếp thông qua các giao diện bằng cách gọi phương thức hoặc truy cập vào thuộc tính của các đối tượng khác.
- Nguyên tắc kiến trúc hướng đối tượng:
  - Trừu tượng.
  - Thành phần.
  - Kế thừa.
  - Đóng gói.
  - Đa hình.
  - Tách biệt.



# | Kiểu kiến trúc hướng đối tượng

- Ưu điểm:
  - Dễ hiểu.
  - Tái sử dụng.
  - Dễ kiểm chứng.
  - Dễ mở rộng
  - Tính liên kết cao.



# | Kiểu kiến trúc hướng dịch vụ (SOA)

- Kiến trúc hướng dịch vụ (Service Oriented Architecture) cho phép ứng dụng cung cấp các chức năng như một tập hợp các dịch vụ và tạo ra các ứng dụng sử dụng dịch vụ.
- Nguyên tắc của kiến trúc SOA:
  - Dịch vụ là tự trị.
  - Dịch vụ liên kết lỏng lẻo.
  - Khả năng tương thích dựa trên chính sách.
- Ưu điểm:
  - Rút ngắn thời gian đưa ra thị trường.
  - Bảo trì hiệu quả.
  - Khả năng thích ứng cao hơn.



## I 2.2 MÔ HÌNH HÓA KIẾN TRÚC

- Kiến trúc là một tập các quyết định thiết kế chính của hệ thống. Một khi các quyết định thiết kế được đưa ra, chúng sẽ được thể hiện bởi một mô hình và quá trình tạo ra mô hình đó được gọi là **mô hình hóa**.
- Các quyết định phải lựa chọn:
  - Các quyết định và các khái niệm cần phải mô hình.
  - Mức độ chi tiết của mô hình những khái niệm.
  - Mức độ hình thức cho mô hình.
- Một số khái niệm kiến trúc cơ bản:
  - **Thành phần**: Thành phần là khối kiến trúc đóng gói một tập chức năng hay dữ liệu của hệ thống và bên ngoài chỉ có thể truy nhập qua một giao diện xác định.
  - **Kết nối**: Kết nối là khối kiến trúc quy định tương tác giữa các thành phần.
  - **Giao diện**: Giao diện là những điểm mà các thành phần và kết nối tương tác với thế giới bên ngoài như các thành phần và kết nối khác.
  - **Cấu hình**: Cấu hình là tập các liên kết giữa các thành phần và kết nối của kiến trúc hệ phần mềm.

# I MỘT SỐ ĐẶC TRƯNG CỦA MÔ HÌNH HÓA KIẾN TRÚC

- Tính chất động và tĩnh của hệ thống.
  - Tĩnh là không thay đổi theo thời gian.
  - Động liên quan đến hành vi của hệ thống theo thời gian.
- Khía cạnh chức năng và phi chức năng.
  - Khía cạnh chức năng liên quan đến những gì hệ thống thực hiện.
  - Khía cạnh phi chức năng liên quan đến cách thức hệ thống thực hiện các chức năng của nó.
- Tính nhập nhằng, đúng đắn và rõ ràng.
  - Tính nhập nhằng: Nếu có nhiều hơn 1 thể hiện.
  - Một mô hình là đúng đắn nếu nó thể hiện đúng, phù hợp với thực tế.
  - Một mô hình là rõ ràng nếu nó cụ thể, chi tiết và chính xác.

# I TÍNH PHỨC TẠP CỦA MÔ HÌNH HÓA

- Các mô hình kiến trúc là những sản phẩm phức tạp. Mô hình kiến trúc cố gắng thể hiện tất cả các quyết định thiết kế về hệ thống mà quan trọng với nhiều bên khác nhau.
- **Khung nhìn (view)** là tập quyết định thiết kế liên quan đến một hay một tập quan tâm chung.
- **Quan điểm (viewpoint)** là cơ sở cho cái nhìn.
- Một số quan điểm về hệ thống:
  - **Quan điểm logic** (logical viewpoint): Thể hiện các thực thể logic trong hệ phần mềm và cách kết nối của chúng.
  - **Quan điểm vật lý** (physical viewpoint): Thể hiện các thực thể vật lý trong hệ thống và cách kết nối của chúng.
  - **Quan điểm triển khai** (deployment viewpoint): Cách các thực thể logic được ánh xạ vào các thực thể vật lý.
  - **Quan điểm tiến trình** (process viewpoint): Thể hiện cách quản lý các luồng trong hệ thống.
  - **Quan điểm hành vi** (behavioral viewpoint): Thể hiện các hành vi của hệ thống.

## I 2.3 KIẾN TRÚC PHẦN MỀM DỰA TRÊN THÀNH PHẦN

- Từ lập trình **hướng đối tượng** đến lập trình **hướng thành phần**:
  - Trong **OOP** (Object-Oriented Programming), các chương trình được xây dựng dựa trên các lớp và các đối tượng.
  - Lập trình **hướng thành phần** (**COP**: Component-Oriented Programming) cho phép các chương trình được xây dựng từ các khối mã máy là các thành phần phần mềm đã được xây dựng sẵn theo các tiêu chuẩn được xác định trước bao gồm giao diện, kết nối, phiên bản và triển khai.
  - **OOP** nhấn mạnh các lớp và các đối tượng, **COP** phát triển phần mềm bằng cách lắp ghép các thành phần và chú trọng đến các giao diện và thành phần.
- Ba đặc trưng chính **COP**: giảm thiểu phức tạp, quản lý thay đổi và tái sử dụng.

# I KIẾN TRÚC PHẦN MỀM DỰA TRÊN THÀNH PHẦN

- Khái niệm **thành phần**: thành phần phần mềm là một khối mã đóng gói một tập các chức năng liên quan và có thể kết hợp với các thành phần khác thông qua giao diện.
- Một **thành phần** có thể được cài đặt và thực thi trong môi trường phần mềm xác định.
- Các công nghệ thành phần gồm: JavaBeans, EJB (Enterprise Java Beans) của Sun Microsystems, COM (Component Object Model), DCOM (Distributed Component Object Model), các thành phần .Net của Microsoft và các thành phần CORBA (Common Object Request Broker Architecture) của nhóm Object Management Group.

# | PHÁT TRIỂN PHẦN MỀM DỰA TRÊN THÀNH PHẦN

- Hướng dẫn thiết kế thành phần:
  - Áp dụng các nguyên lý thiết kế SOLID:
    - Nguyên lý đơn nhiệm: Một lớp chỉ nên gán một trách nhiệm.
    - Nguyên lý đóng/mở: Các lớp phải mở rộng được mà không đòi hỏi sửa đổi.
    - Nguyên lý thay thế Liskov: Các kiểu con phải thay thế được cho các kiểu cơ bản.
    - Nguyên lý giao tiếp tách biệt: Các lớp nên có những giao tiếp riêng cho những đối tác có yêu cầu khác nhau.
    - Nguyên lý đảo phụ thuộc: Phụ thuộc giữa các lớp nên được thay thế bởi các lớp trừu tượng để cho phép thiết kế kiểu top-down.
  - Các thành phần thiết kế phải kết hợp chặt chẽ với nhau.
  - Thành phần không nên dựa vào chi tiết bên trong các thành phần khác.
  - Hiểu cách giao tiếp giữa các thành phần với nhau.
  - Đảm bảo mã là trừu tượng với logic ứng dụng, dễ mở rộng sau này.
  - Tái sử dụng được, thay thế được, mở rộng được.



## I 2.4 MÔ HÌNH HÓA KIẾN TRÚC VỚI UML

- Ngôn ngữ mô hình hóa thống nhất **UML** (Unified Modeling Language) là bộ ký hiệu với biểu đồ quen thuộc nhất để tạo ra bản thiết kế phần mềm.
- **UML** sử dụng một hệ thống ký hiệu thống nhất biểu diễn các Phần tử mô hình (**model elements**). Tập hợp các phần tử mô hình tạo thành các Sơ đồ UML (UML diagrams).
- Có các loại sơ đồ UML chủ yếu sau:
  - Sơ đồ lớp (Class Diagram)
  - Sơ đồ đối tượng (Object Diagram)
  - Sơ đồ tình huống sử dụng (Use Cases Diagram)
  - Sơ đồ trình tự (Sequence Diagram)
  - Sơ đồ cộng tác (Collaboration Diagram hay là Composite Structure Diagram)
  - Sơ đồ trạng thái (State Machine Diagram)
  - Sơ đồ thành phần (Component Diagram)
  - Sơ đồ hoạt động (Activity Diagram)
  - Sơ đồ triển khai (Deployment Diagram)
  - Sơ đồ gói (Package Diagram)
  - Sơ đồ liên lạc (Communication Diagram)
  - Sơ đồ tương tác (Interaction Overview Diagram - UML 2.0)
  - Sơ đồ phối hợp thời gian (Timing Diagram - UML 2.0)

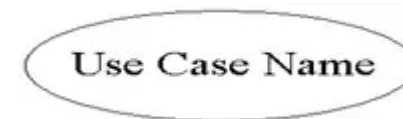
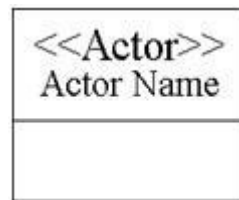
# | Sơ đồ Use case (Use Case Diagram)

- Một sơ đồ **Use case** chỉ ra một số lượng các tác nhân ngoại cảnh và mối liên kết của chúng đối với Use case mà hệ thống cung cấp. Một Use case là một lời miêu tả của một chức năng mà hệ thống cung cấp.
- Các Use case định nghĩa các yêu cầu về mặt chức năng đối với hệ thống:
  - **Hệ thống**: hệ thống biểu diễn ranh giới giữa bên trong và bên ngoài của một chủ thể trong phần mềm chúng ta xây dựng.
  - **Tác nhân (actor)**: là người dùng của hệ thống, một tác nhân có thể là một người dùng thực hoặc các hệ thống máy tính khác có vai trò nào đó trong hoạt động của hệ thống.



Ký hiệu của Tác nhân

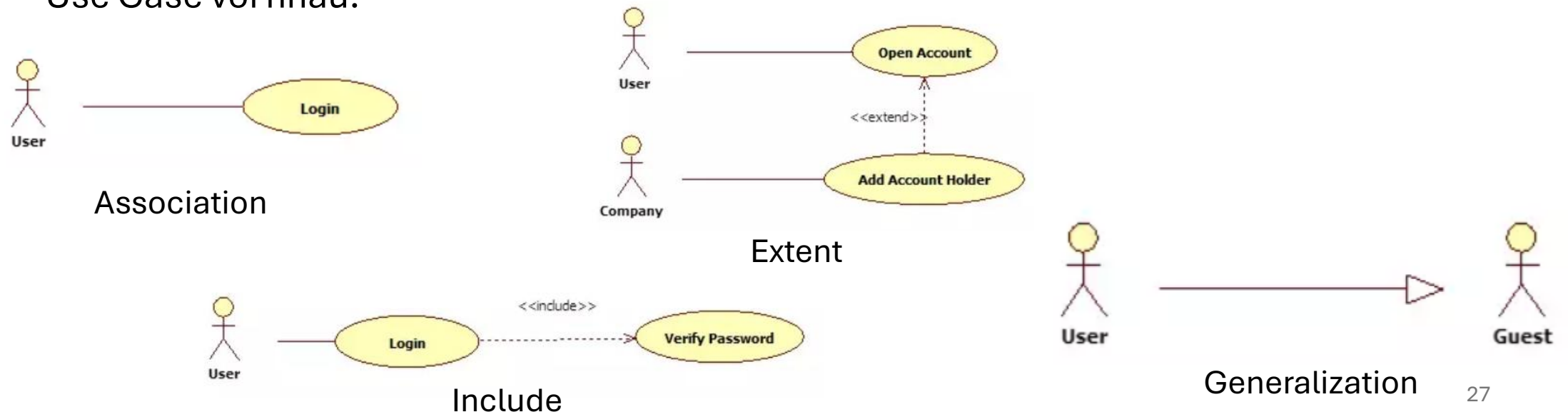
Hoặc



Ký hiệu của Use case

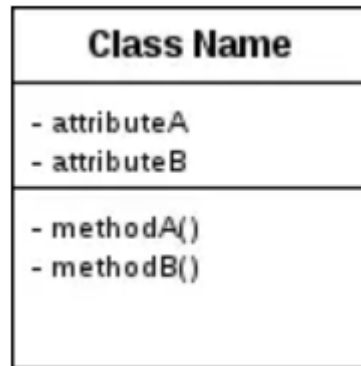
# | Sơ đồ Use case (Use Case Diagram)

- Mỗi quan hệ giữa các Use Case:
  - **Association**: thường được dùng để mô tả mối quan hệ giữa Actor và Use Case và giữa các Use Case với nhau.
  - **Include**: là quan hệ giữa các Use Case với nhau, nó mô tả việc một Use Case lớn được chia ra thành các Use Case nhỏ để dễ cài đặt (module hóa) hoặc thể hiện sự dùng lại.
  - **Extent**: Extend dùng để mô tả quan hệ giữa 2 Use Case. Quan hệ Extend được sử dụng khi có một Use Case được tạo ra để bổ sung chức năng cho một Use Case có sẵn và được sử dụng trong một điều kiện nhất định nào đó.
  - **Generalization**: được sử dụng để thể hiện quan hệ thừa kế giữa các Actor hoặc giữa các Use Case với nhau.



# | Sơ đồ lớp (Class Diagram)

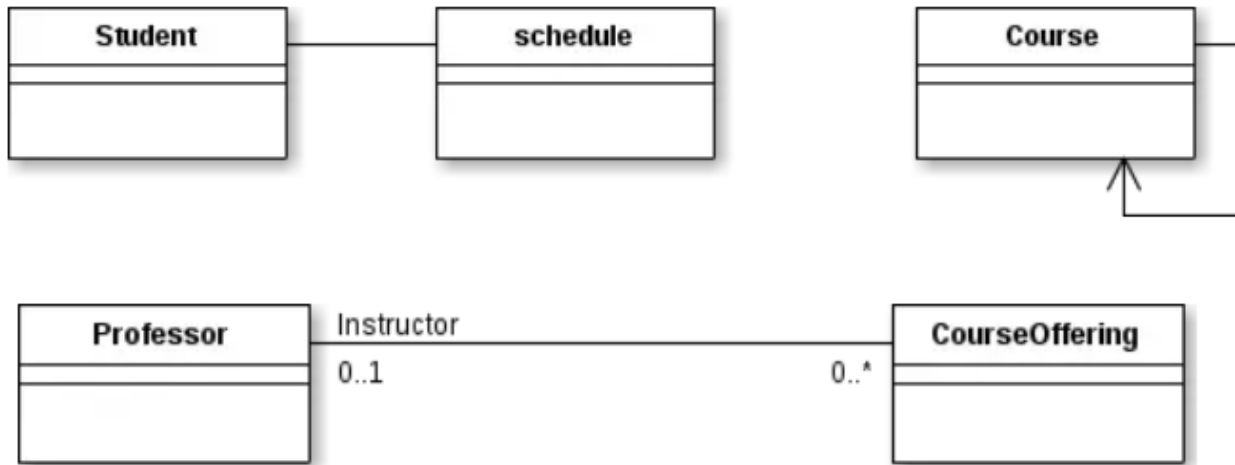
- Một **biểu đồ lớp** chỉ ra cấu trúc tĩnh của các lớp trong hệ thống. Các lớp là đại diện cho các “đối tượng” được xử lý trong hệ thống. Các lớp có thể quan hệ với nhau trong nhiều dạng thức.
  - Liên kết (associated - được nối kết với nhau),
  - Phụ thuộc (dependent - một lớp này phụ thuộc vào lớp khác),
  - Chuyên biệt hóa (specialized - một lớp này là một kết quả chuyên biệt hóa của lớp khác),
  - Đóng gói (packaged - hợp với nhau thành một đơn vị).
- Các thành phần của lớp:
  - Tên lớp
  - Các thuộc tính
  - Các phương thức.



# | Sơ đồ lớp (Class Diagram) - Liên kết giữa các lớp

- Liên kết (**Association**):

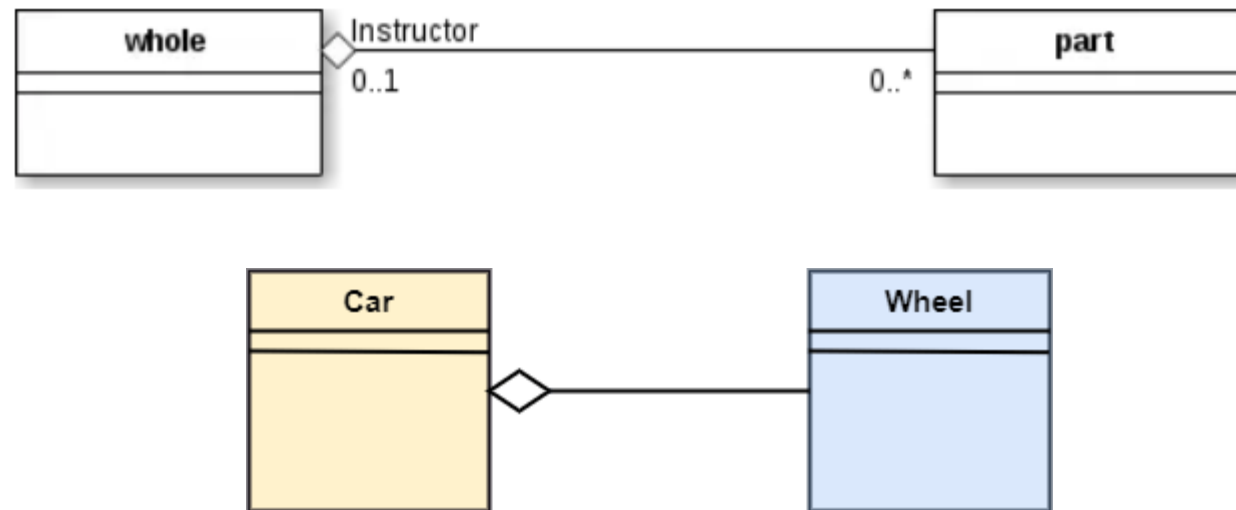
- Mỗi liên hệ ngữ nghĩa giữa hai hay nhiều lớp chỉ ra sự liên kết giữa các thể hiện của chúng.
- Bộ số quan hệ: là số lượng thể hiện của một lớp liên quan tới một thể hiện của lớp khác. Với mỗi liên kết, có hai bộ số quan hệ cho hai đầu của liên kết.



| Biểu diễn | Ý nghĩa                   |
|-----------|---------------------------|
|           | Unspecified               |
| 1         | chính xác 1               |
| 0..*      | 0 hoặc nhiều              |
| *         | 0 hoặc nhiều              |
| 1..*      | 1 hoặc nhiều              |
| 0..1      | 0 hoặc 1                  |
| 2..4      | Specified Range           |
| 2, 4..6   | Multiple, Disjoint Ranges |

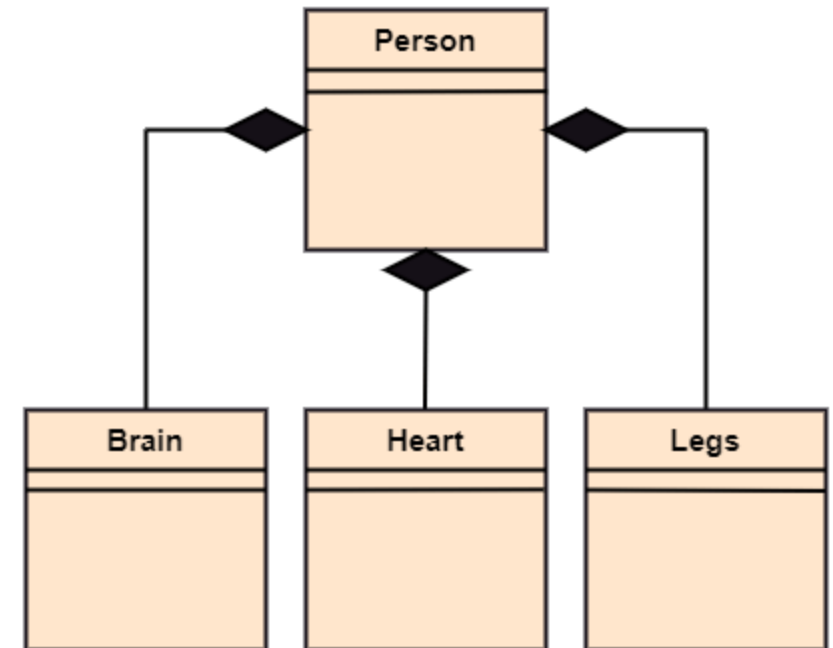
# I Sơ đồ lớp (Class Diagram) - Liên kết giữa các lớp

- Kết tập (**Aggregation**): Là một dạng đặc biệt của liên kết mô hình hóa mối quan hệ toàn thể-bộ phận (whole-part) giữa đối tượng toàn thể và các bộ phận của nó.
- Kết tập là mối quan hệ “là một phần” (“is a part-of”).
- Bội số quan hệ được biểu diễn giống như các liên kết khác.



# | Sơ đồ lớp (Class Diagram) - Liên kết giữa các lớp

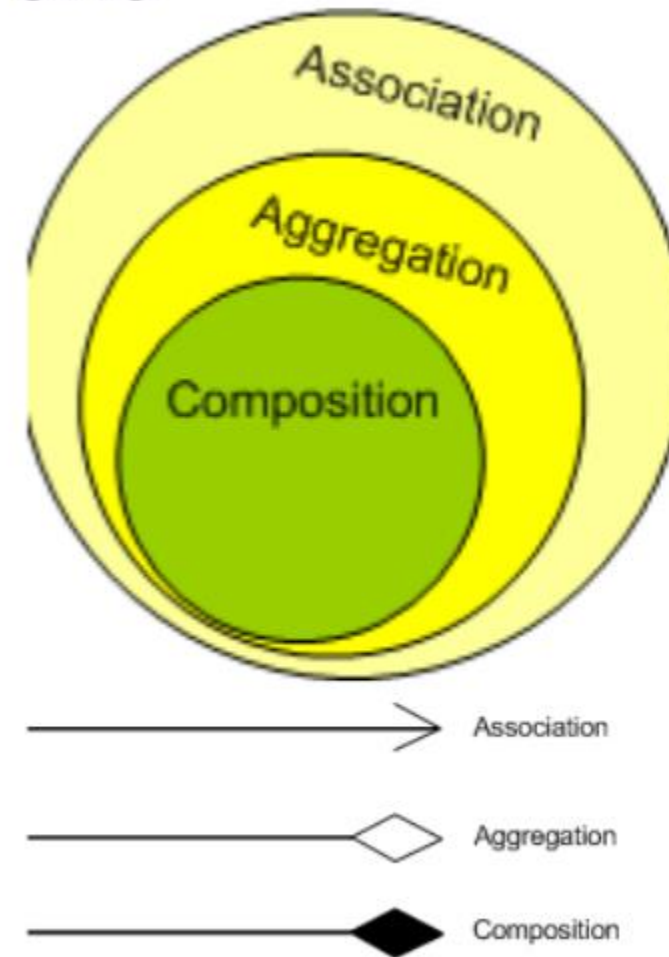
- Cấu thành (**Composition**) là: Một dạng của kết tập với quyền sở hữu mạnh và các vòng đời trùng khớp giữa hai lớp.
- Whole sở hữu Part, tạo và hủy Part.
- Part bị bỏ đi khi Whole bị bỏ, Part không thể tồn tại nếu Whole không tồn tại.



# | Sơ đồ lớp (Class Diagram) - Liên kết giữa các lớp

## Association, Aggregation and Composition

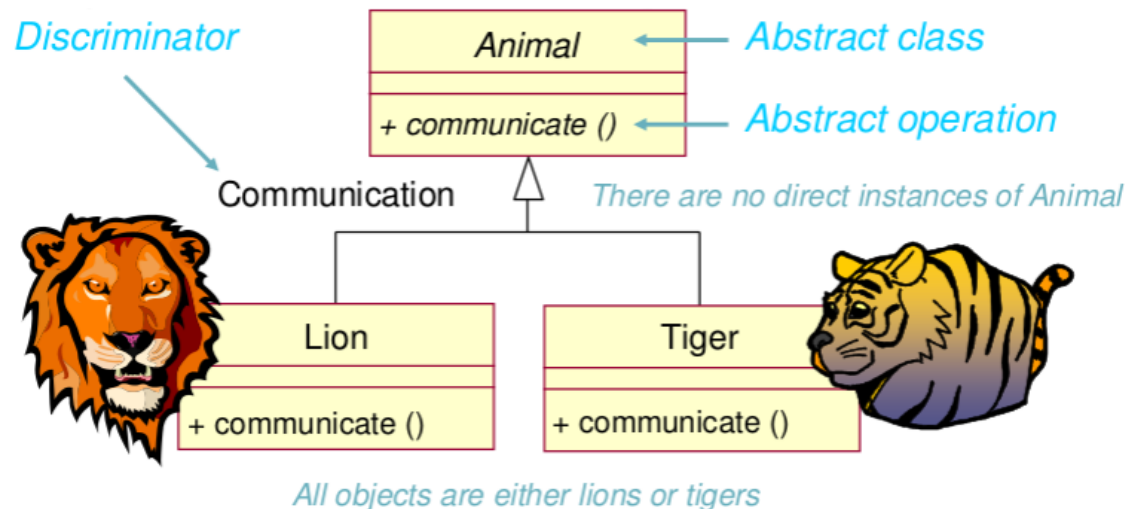
- Mỗi quan hệ giữa các lớp (relationship)
  - Liên kết (Association)
    - Sử dụng (use-a)
  - Kết tập (Aggregation)
    - Strong association
    - has-a/is-a-part
  - Hợp thành (Composition)
    - Strong aggregation
    - Share life-time





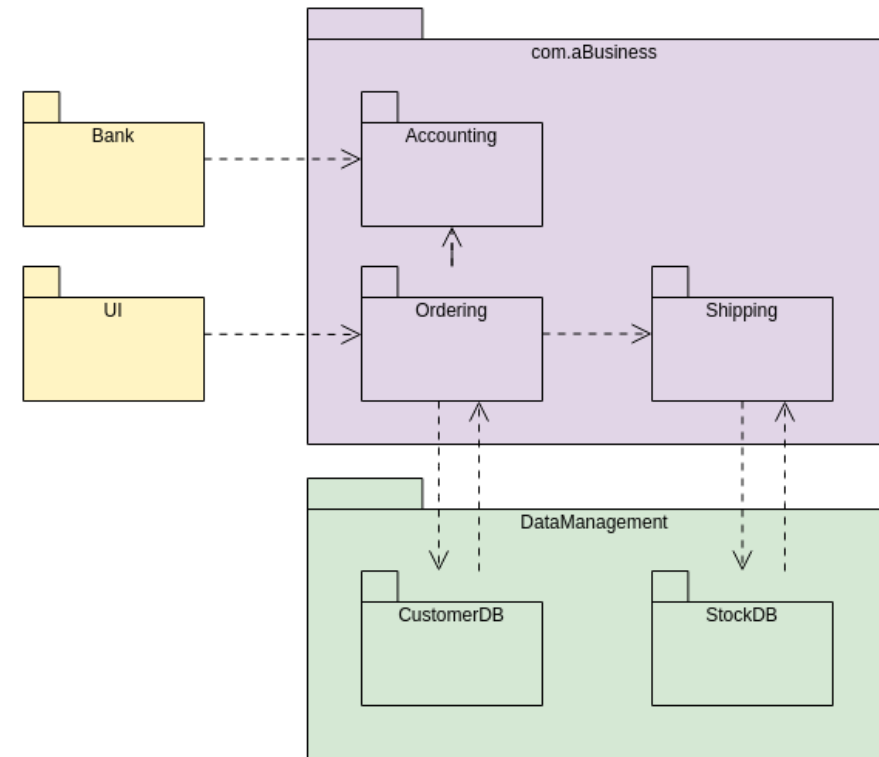
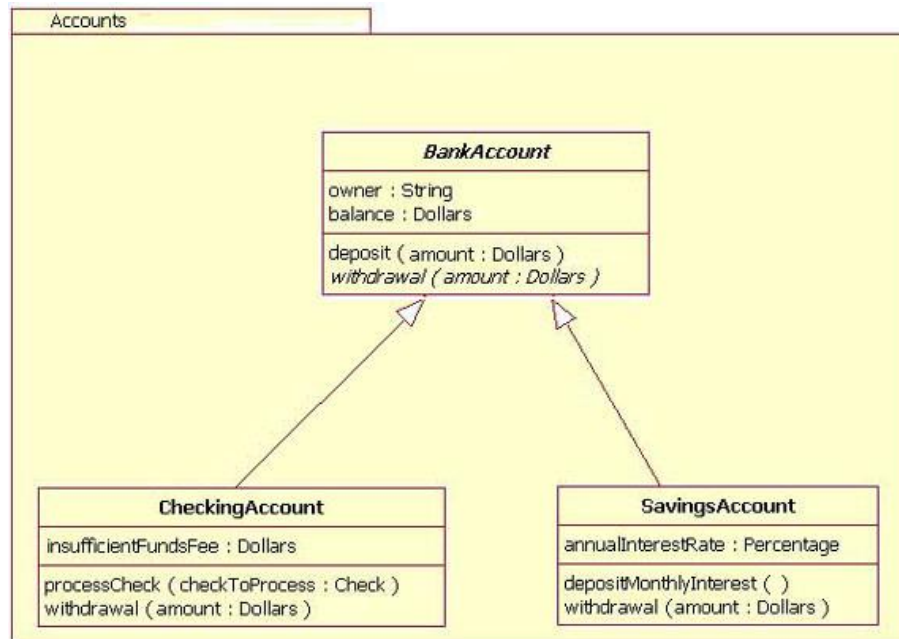
# I Sơ đồ lớp (Class Diagram) - Liên kết giữa các lớp

- Tổng quát hóa (**Generalization**): Mỗi quan hệ giữa các lớp trong đó một lớp chia sẻ cấu trúc và/hoặc hành vi với một hoặc nhiều lớp khác.
- Xác định sự phân cấp về mức độ trừu tượng hóa trong đó lớp con kế thừa từ một hoặc nhiều lớp cha.
  - Đơn kế thừa (Single inheritance)
  - Đa kế thừa (Multiple inheritance)
- Là mối liên hệ “là một loại” (“is a kind of”).
- Lớp trừu tượng và lớp cụ thể (Abstract and Concrete Class).



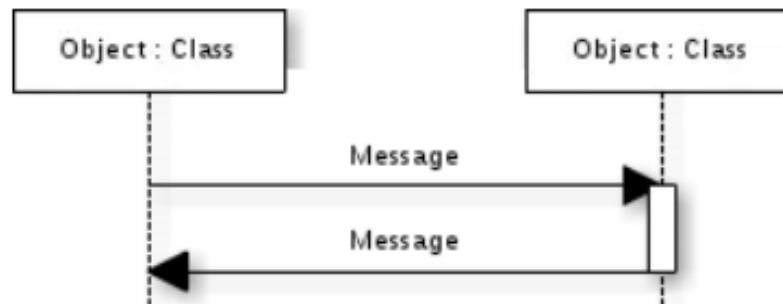
# | Sơ đồ gói (Package Diagram)

- Các **gói** trong **UML** được sử dụng để nhóm các phần tử và cung cấp không gian tên cho các phần tử được nhóm. Một gói có thể chứa các gói khác, do đó cung cấp một tổ chức phân cấp các gói.



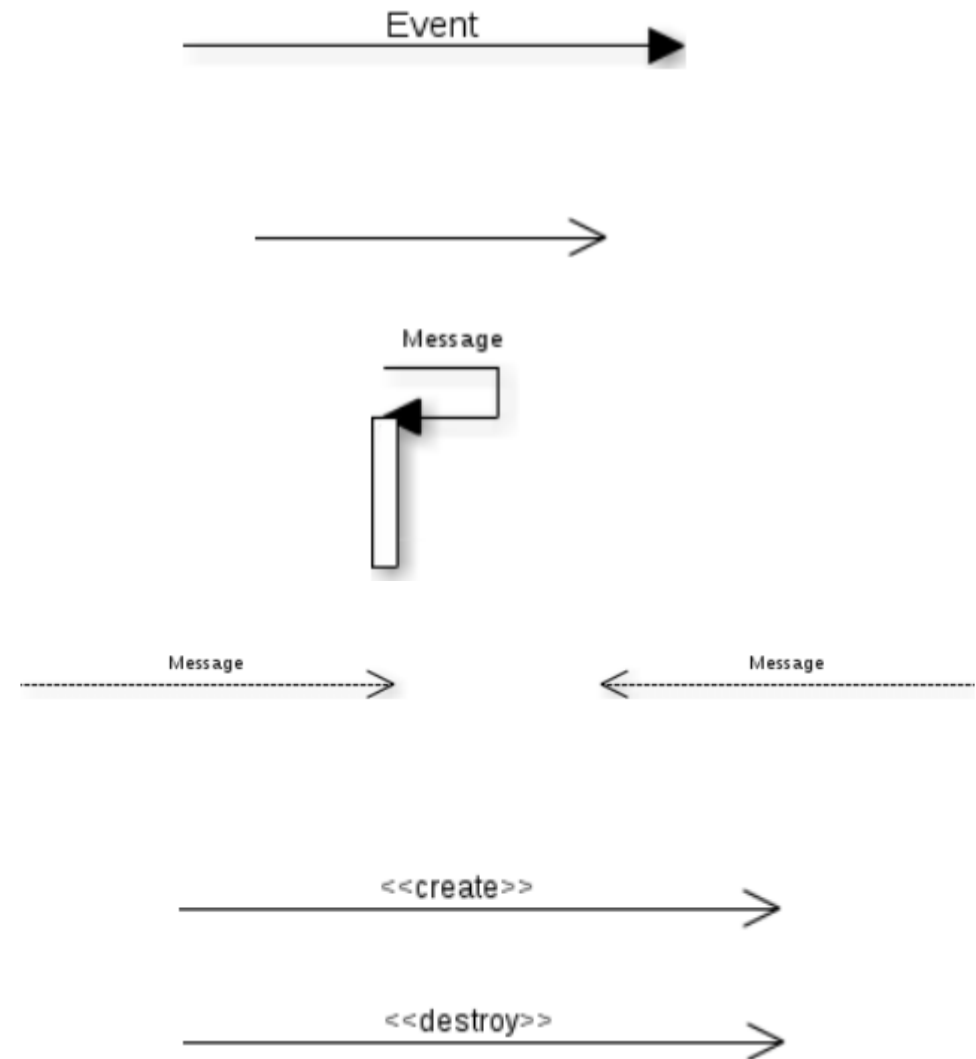
# | Sơ đồ tuần tự (Sequence Diagram)

- **Sơ đồ tuần tự** là sơ đồ dùng để xác định các trình tự diễn ra sự kiện của một nhóm đối tượng nào đó. Nó miêu tả chi tiết các thông điệp được gửi và nhận giữa các đối tượng đồng thời cũng chú trọng đến việc trình tự về mặt thời gian gửi và nhận các thông điệp đó.
- Các thành phần của sơ đồ tuần tự:
  - **Đối tượng** (object or class): biểu diễn bằng các hình chữ nhật.
  - **Đường đời đối tượng** (Lifelines): biểu diễn bằng các đường gạch rời thẳng đứng bên dưới các đối tượng.
  - **Thông điệp** (Message): biểu diễn bằng các đường mũi tên.
  - **Xử lý bên trong đối tượng** (biểu diễn bằng các đoạn hình chữ nhật rỗng nối với các đường đời đối tượng).

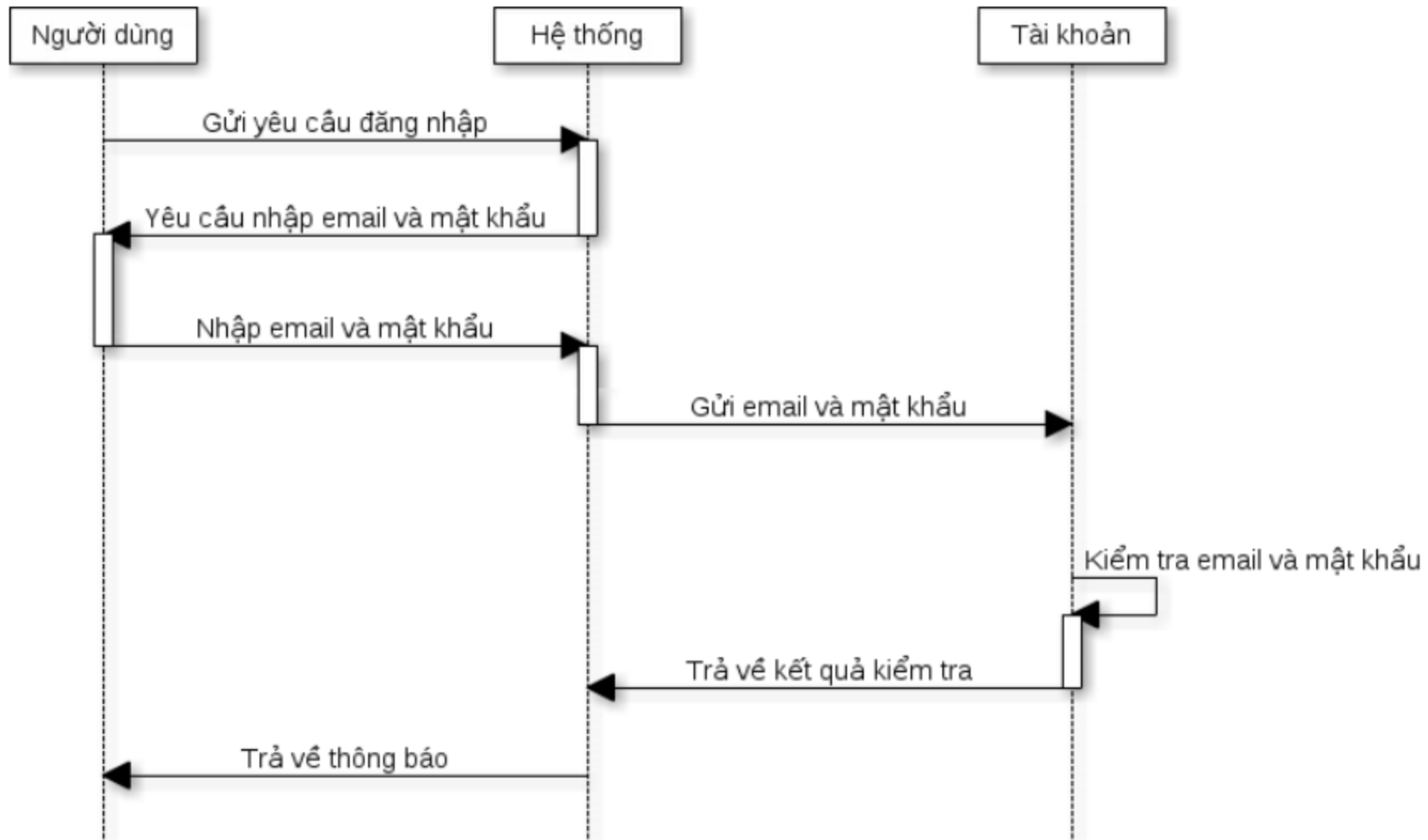


# | Sơ đồ tuần tự (Sequence Diagram) – Các loại thông điệp

- **Thông điệp đồng bộ (Synchronous Message)** Thông điệp đồng bộ cần có một request trước hành động tiếp theo.
- **Thông điệp không đồng bộ (Asynchronous Message)** Thông điệp không đồng bộ không cần có một request trước hành động tiếp theo.
- **Thông điệp chính mình (Self Message)** Là thông điệp mà đối tượng gửi cho chính nó để thực hiện các hàm nội tại.
- **Thông điệp trả lời hoặc trả về (Reply or Return Message)**: Là thông điệp trả lời lại khi có request hoặc sau khi kiểm tra tính đúng đắn của một điều kiện nào đó.
- **Thông điệp tạo mới (Create Message)**: Là thông điệp được trả về khi tạo mới một đối tượng.
- **Thông điệp xóa (Delete Message)**: Là thông điệp được trả về khi xóa một đối tượng



# | Sơ đồ tuần tự (Sequence Diagram) – Ví dụ

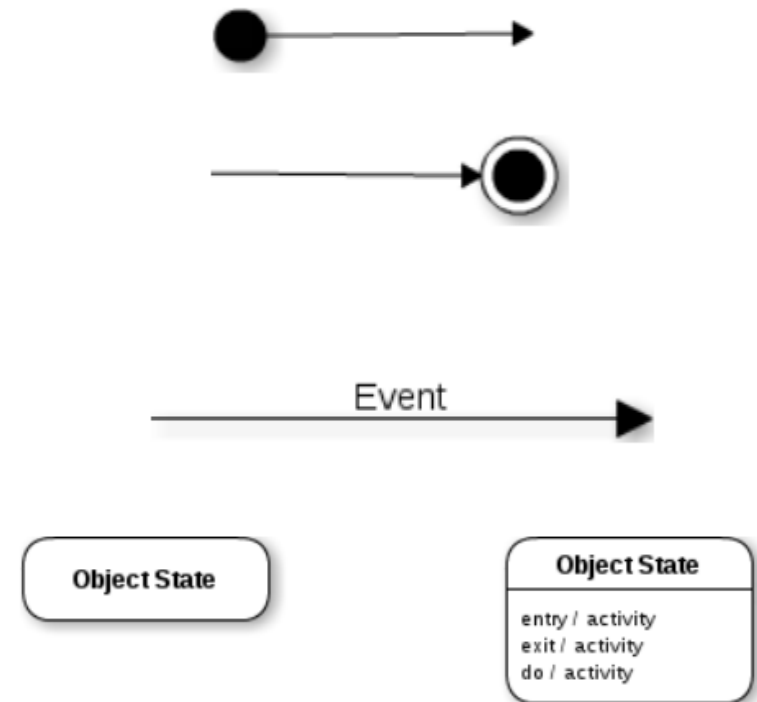


# | Sơ đồ trạng thái (State Diagram)

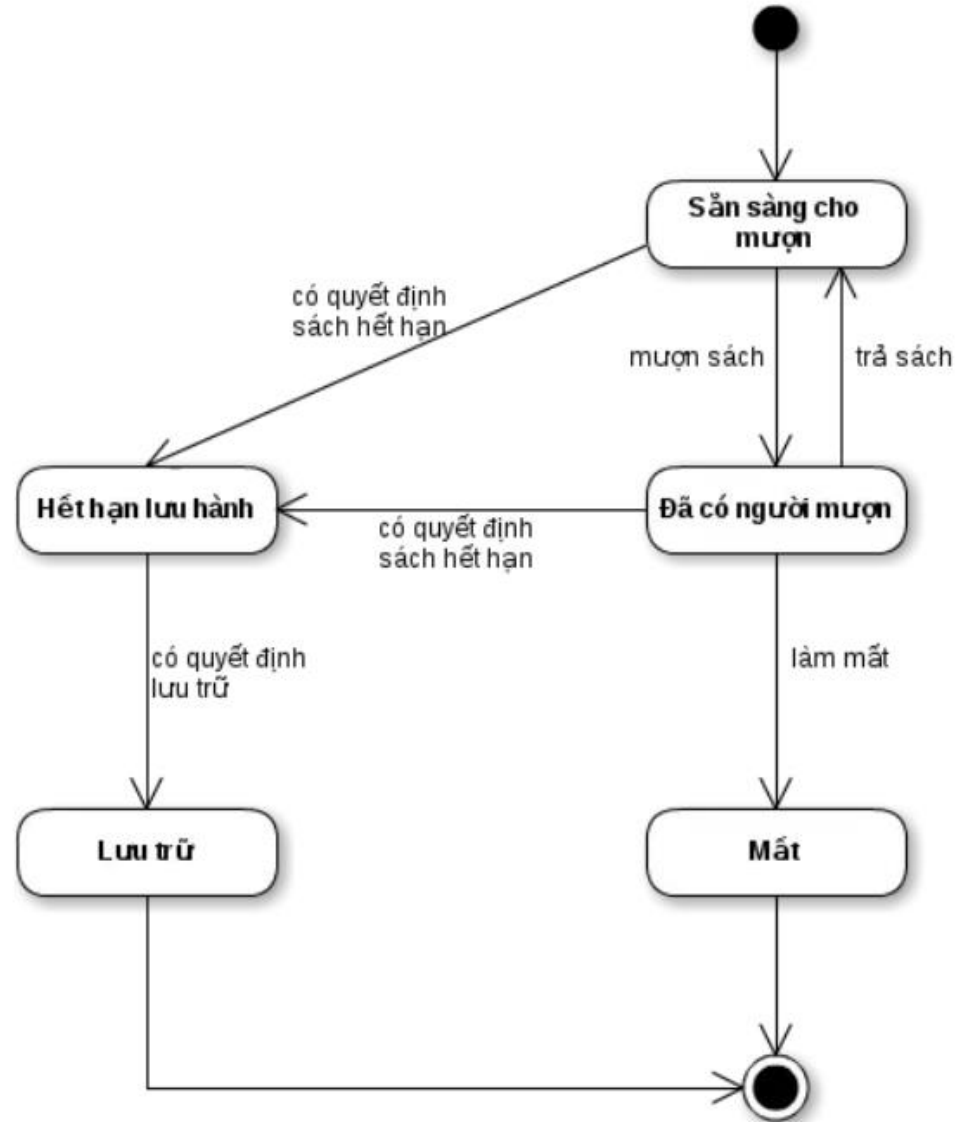
- **Sơ đồ trạng thái** là dạng sơ đồ mô tả các trạng thái có thể có và sự chuyển đổi giữa các trạng thái đó khi có các sự kiện tác động của một đối tượng.
- Các thành phần của Sơ đồ trạng thái:
  - **Trạng thái bắt đầu**: (Initial State).
  - **Trạng thái kết thúc**: (Final State):

Trong biểu đồ, đường mũi tên chỉ ra sự biến đổi từ một trạng thái sang trạng thái khác.

  - **Sự kiện** (Event) hoặc **Chuyển đổi** (Transition).
  - **Trạng thái đối tượng** (State)

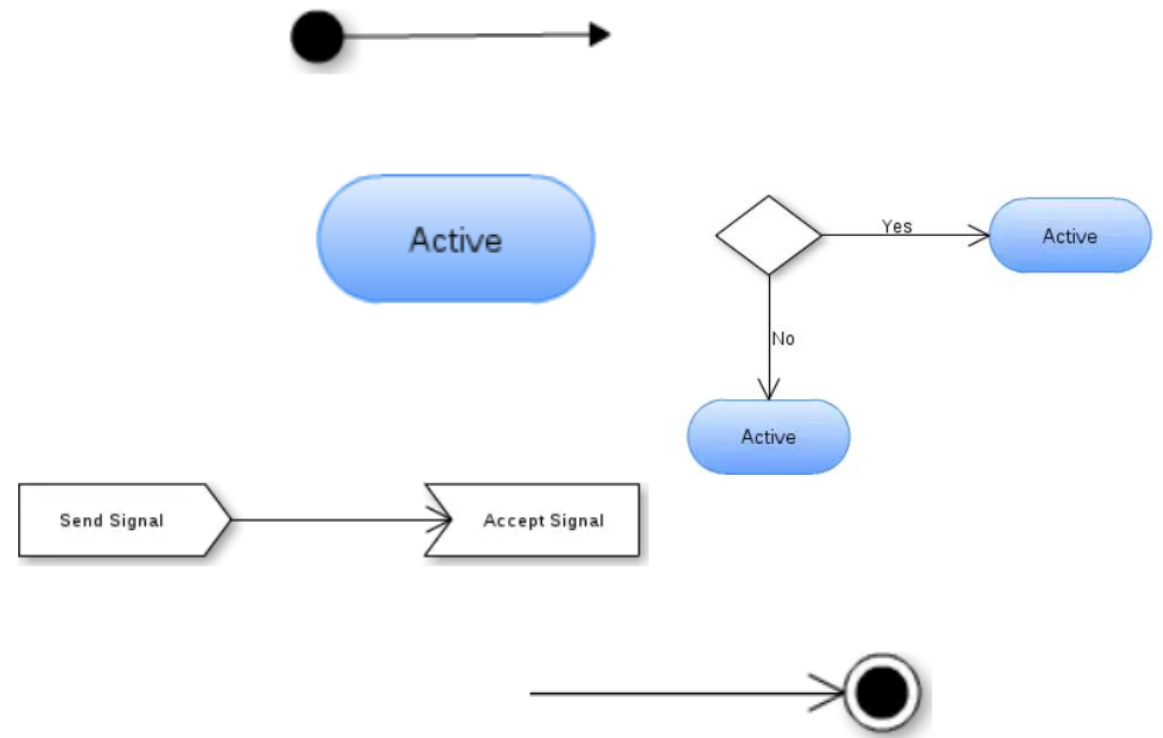


# | Sơ đồ trạng thái (State Diagram) – Ví dụ



# | Sơ đồ hoạt động (Activity Diagram)

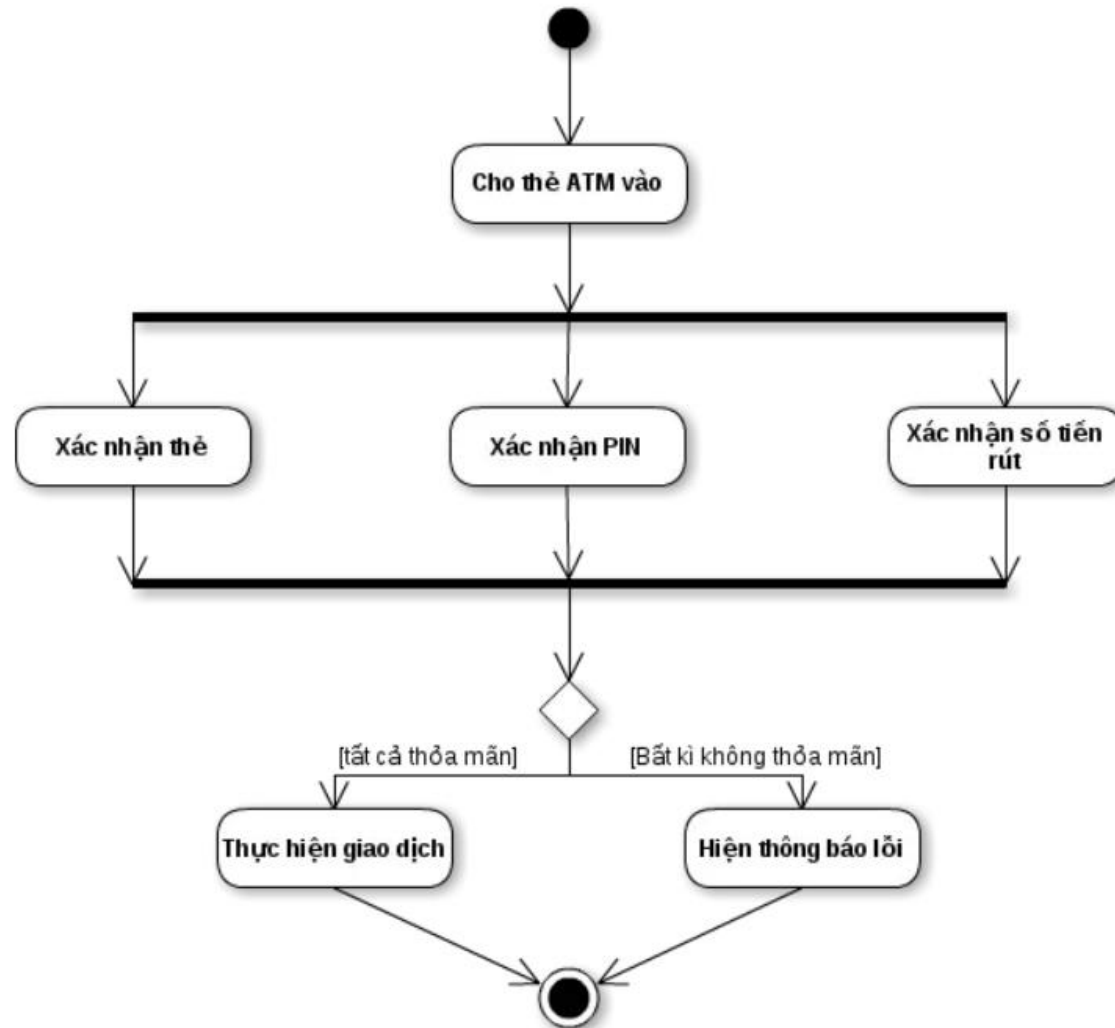
- **Sơ đồ hoạt động** là sơ đồ mô tả các bước thực hiện, các hành động, các nút quyết định và điều kiện rẽ nhánh để điều khiển luồng thực hiện của hệ thống.
- Các thành phần:
  - **Trạng thái khởi tạo** hoặc điểm bắt đầu (Initial State or Start Point).
  - **Hoạt động** hoặc **trạng thái hoạt động** (Activity or Action State).
  - Nút quyết định và rẽ nhánh.
  - Luồng hành động.
  - **Gửi và nhận tín hiệu** (Sent and Received Signals).
  - **Trạng thái kết thúc** hoặc điểm cuối (Final State or End Point).





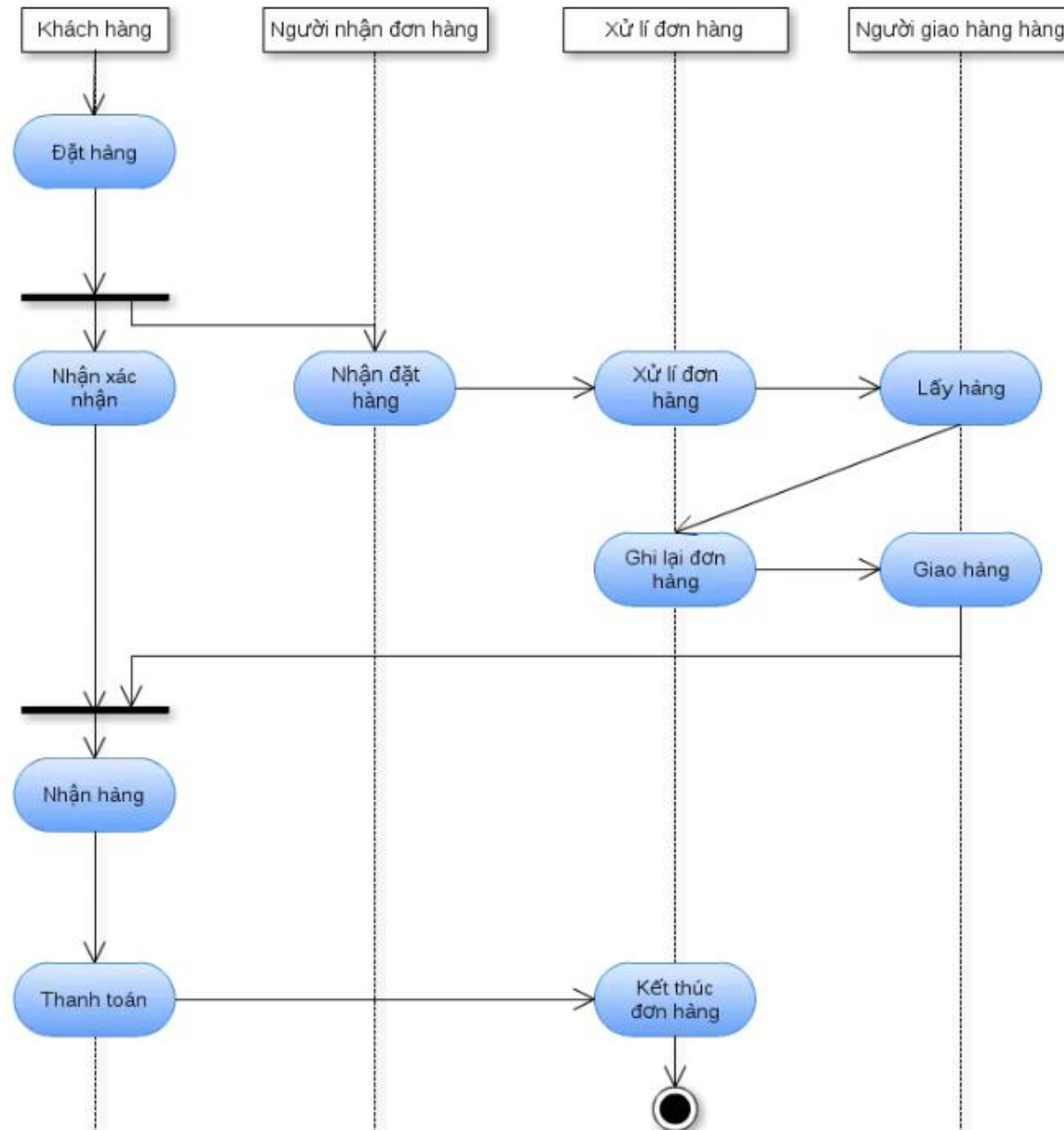
# | Sơ đồ hoạt động (Activity Diagram) – Ví dụ

- Sơ đồ hoạt động rút tiền tại cây ATM:



# | Sơ đồ hoạt động (Activity Diagram) – Ví dụ

Sơ đồ hoạt động đặt hàng:



## | BÀI TẬP

- Sử dụng UML để mô hình hóa một hệ thống bất kì, ví dụ quản lý thư viện, bán hàng online, quản lý công việc, đăng ký học tín chỉ ...